

Summary

With the rapid development of artificial intelligence technology, humanoid robots that appear only in fiction movies are now gradually turning from imagination to reality. They are capable of doing both simple repetitive work and complicated tasks that involve more body movement and interaction. Our team has been invited by the Intelligent Mechanized Manufacturing Corporation (IMMC) to assist in developing a rope-jumping function for their latest humanoid robot, FlexRobot.

Our team first analyzed the robot's motion in the rope-jumping process. This includes relevant variables to describe the motion when the robot is on the ground and in the air.

We then started our construction of the mathematical model to describe the trajectory shape of the rope movement. We regarded the jump rope as a **catenary**, since its shape satisfies the basic form and identities of a catenary. We also noticed that the shape of a rotating catenary can be described using the **Jacobi Elliptic Function (sn)**. So, we investigated variables and parameters involved in the sn function and constructed their respective functions with respect to time.

Eventually, the function can be drawn in MATLAB and our model could display the shape of the rope at any moment of the rope-jumping process both two-dimensionally and three-dimensionally. The 2-D and 3-D results are provided in this paper.

Moreover, our team also made some relevant calculations. These include the range of initial jumping velocity, the range of jumping time, and the range of rope length. These numerical analyses can help us have a better understanding of the rope-jumping process.

Keywords: Rope-Jumping Motion, Catenary Equation, Jacobi Elliptic Function, Humanoid Robot Application

Contents

1	Introduction	3
1.1	Background	3
1.2	Problem Restatement	3
2	Assumptions and Justifications	4
3	Variables and Parameters	4
4	Robot and Rope-Jumping Motion Analysis	5
4.1	Variable Definition	5
4.2	Motion Description	5
4.2.1	On the Ground	5
4.2.2	In the Air	5
5	Jump Rope Trajectory Model	7
5.1	Model Overview	7
5.2	Catenary	7
5.3	Jacobi Elliptic Function (sn)	8
5.4	Model Construction	8
5.4.1	Jumping Height (Δh)	8
5.4.2	Hand Distance (a)	10
5.4.3	Rope Height (h)	10
5.4.4	Other parameters	10
5.5	Model Results	11
5.5.1	2-D Results	12
5.5.2	3-D Results	13
6	Relevant Calculation	13
6.1	Range of Initial Jumping Velocity v_0	13
6.2	Range of Jumping Time (t_j)	13
6.3	Range of Rope Length (l)	14
7	Arm Length (d) Analysis	15
7.1	Influence on Rope Length (l)	15
7.2	Influence on Rope Shape ($y(x)$)	15
8	Conclusions	17
	Appendices	18

1 Introduction

1.1 Background

Due to the rapid development of artificial intelligence technology, humanoid robots that appeared only in fiction movies are now gradually turning from imagination to reality. Industries around the world believe that robots will be the next direction of artificial intelligence technology, especially humanoid robots.

Currently, humanoid robots possess lots of potential in various fields, in which they can be used in many ways. Among them the most intriguing and promising are household humanoid robots, where they can do some everyday tasks that involve repetition of work, such as cleaning the room and dishwashing.

Besides, humanoid robots are also capable of doing some complicated tasks that involve more body movement and coordination. For example, an artificial intelligence manufacturing company called Intelligent Mechanized Manufacturing Corporation (IMMC) is currently developing a rope-jumping function for its latest version of humanoid robot, FlexRobot. This function aims to transform the action of rope-jumping from humans to robots. Thus, our team is honored to be invited to conduct research to assist in developing this function.

1.2 Problem Restatement

Below is a brief description of the motion of FlexRobot according to the problem, which will be used in our team's research on the rope-jumping function development.

- The change in leg length at takeoff can be ignored.
- The width of the legs remains constant during the jumping process.
- The rope swinging movement is controlled by electric motors in the arms, which can achieve a stable and constant rope swinging angular velocity.
- With the assistance of a gyroscope, the height of the arms relative to the ground is unchanged during both the standing and takeoff process.
- To prevent the rope from irregularly bouncing back after colliding with the ground, the robot is designed to make the lower end of the rope just pass through the ground level when jumping rope.

Below are the tasks that our team are required to accomplish.

1) Under the condition that the width between the two arms is fixed, establish a mathematical model to describe the trajectory shape of the rope movement.

2) Calculate the range of the rope length, the magnitude of the initial velocity for each jump, and the range of the jumping time, based on the previously established trajectory model and the following conditions. The robot has a height of 1.70 m, with arms at a height of 1.0 m above the ground. The width between the two arms ranges from 1.0 to 1.2 m. The rope-swinging angular velocity is one circle per second, and the width of the legs is 0.5 m

2 Assumptions and Justifications

According to relevant papers and the actual situation, we make the following assumptions:

- 1) Air resistance is negligible since the shape of the rope is approximated to have zero volume.
- 2) We assume that gravity does not cause a difference in the shape of the rope when it rotates to the highest point and when it is at the bottom.
- 3) We assume that the rope does not interact with the robot's leg when the robot is at its highest point, with arms clinging to its body.

3 Variables and Parameters

Below are the variables and parameters that will be used in our team's model construction and value calculations.

Variable Symbol	Explanation
x	x -coordinate
y	y -coordinate
t	Time
t_j	The time of each jump from takeoff to landing (jumping time)
a	Distance from the hand to the y -axis (hand distance)
a'	Distance from the hand to the shoulder
h	Distance from the rope's bottom or top to the x -axis (rope height)
Δh	Jumping height
d	Arm length
l	Rope length
v_0	Initial jumping velocity
g	Gravitational acceleration
sn	Jacobi Elliptic Function
L	A length parameter in sn
k	A parameter in sn
B	A parameter in sn that affects horizontal shift
S	A parameter in sn that affects vertical shift

Table 1: Variables and Parameters

4 Robot and Rope-Jumping Motion Analysis

4.1 Variable Definition

We define a to be the horizontal distance of one of the robot's "hands" to the vertical axis of symmetry of the robot, a' to be the horizontal distance of one of the robot's "hands" to its corresponding shoulder. Since the width of the legs, also the width of the body, is 0.5 m. We call it the "hand distance."

$$a' = a + 0.25 \quad (1)$$

We define h to be the distance of the rope's bottom or top end to the axis of rotation of the rope. We call it the "rope height."

We define Δh to be the robot's vertical jumping height.

We define d to be the length of each arm and choose the width between the two arms to be 1 m. This means that $d = 0.25 \text{ m}$.

4.2 Motion Description

4.2.1 On the Ground

Each jump begins when the robot is on the ground, $\Delta h = 0$, and the arms are in a horizontal position. We denote the horizontal line that connects the two arms to be the **x -axis**. The x -axis is the fixed axis of rotation of the rope, which is always 1 m above the ground. At this moment, Δh and h are at their minimum, a and a' are at their maximum.

The shape of the rope (a half loop-like curve) should be the fattest, with the largest horizontal span and the smallest vertical span.

4.2.2 In the Air

When the robot is in the air, $\Delta h > 0$. In each vertical jump, the robot should always be symmetric with respect to a vertical line that passes through the center of the robot, which we denote it with the **y -axis**. Since the shoulder of the robot moves up and down and does not shift horizontally, the vertical jumping height, Δh , is also equal to the vertical shift of the robot's shoulder from its original position, where the original position corresponds to when the robot is standing on the ground.

According to the brief description of the robot's motion in part 1.2, the height of the arms relative to the ground is unchanged. Intuitively, this statement does not make sense since the height

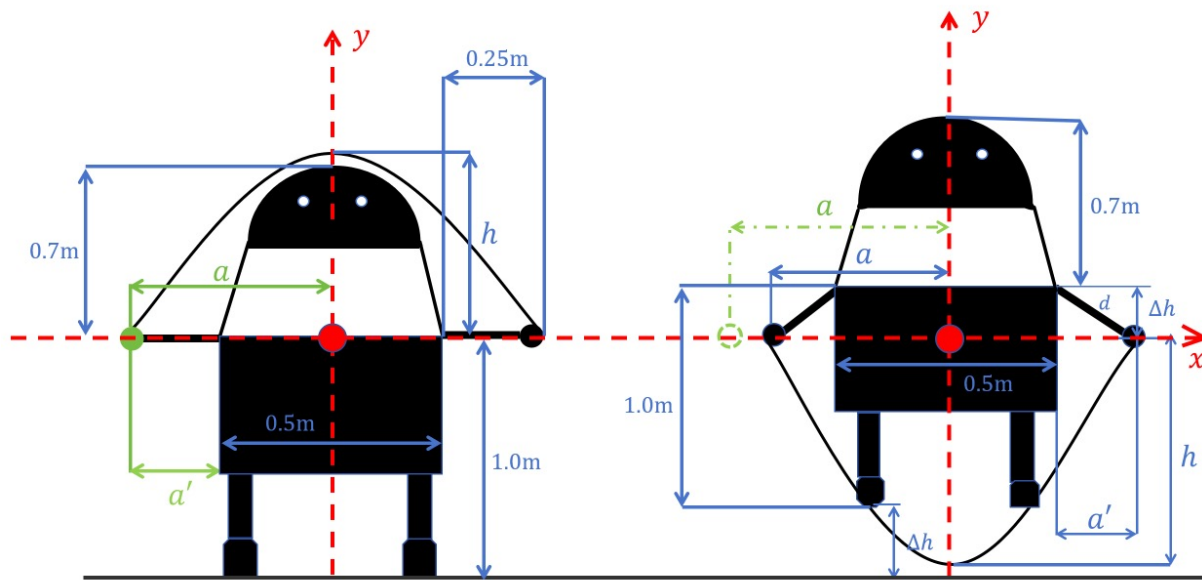


Figure 1: Robot's Motion Both on the Ground and in the Air

of the arms should increase when the robot rises. However, our team resolves this misconception by considering that the height of the robot's hands, not the whole arm, remains constant, and the arms shift from a horizontal to an oblique position, which is shown in **Figure 1**.

Since the length of the arm d remains constant, Δh , a' and d forms a right triangle, which is shown in the right **Figure 1**. Thus, two hands of the robot should move horizontally inward toward the body along the x -axis so that can decrease its value to let Δh increase.

When the robot jumps to its highest position, Δh and h are at their maximum and the rope should be directly below the robot. This means that a' and a are at their minimum. At this moment, the shape of the rope (a half loop-like curve) should be the thinnest, with the smallest horizontal span and the largest vertical span. Since the rope just passes through the ground level, $h = 1$ m. On the contrary, $h < 1$ m when the robot is not at its highest point.

Thus, we can conclude a phenomenon during each jump. When Δh is increasing, h increases, a and a' decreases, vice versa.

5 Jump Rope Trajectory Model

5.1 Model Overview

Our team regards the shape of a jump rope as a kind of **catenary**. With further investigation of relative variables that describe the robot's motion, the shape of the curve is eventually given by the **Jacobi Elliptic Function (sn)**, which can be drawn in MATLAB.

Figure 2 below shows the general outline of our Model.

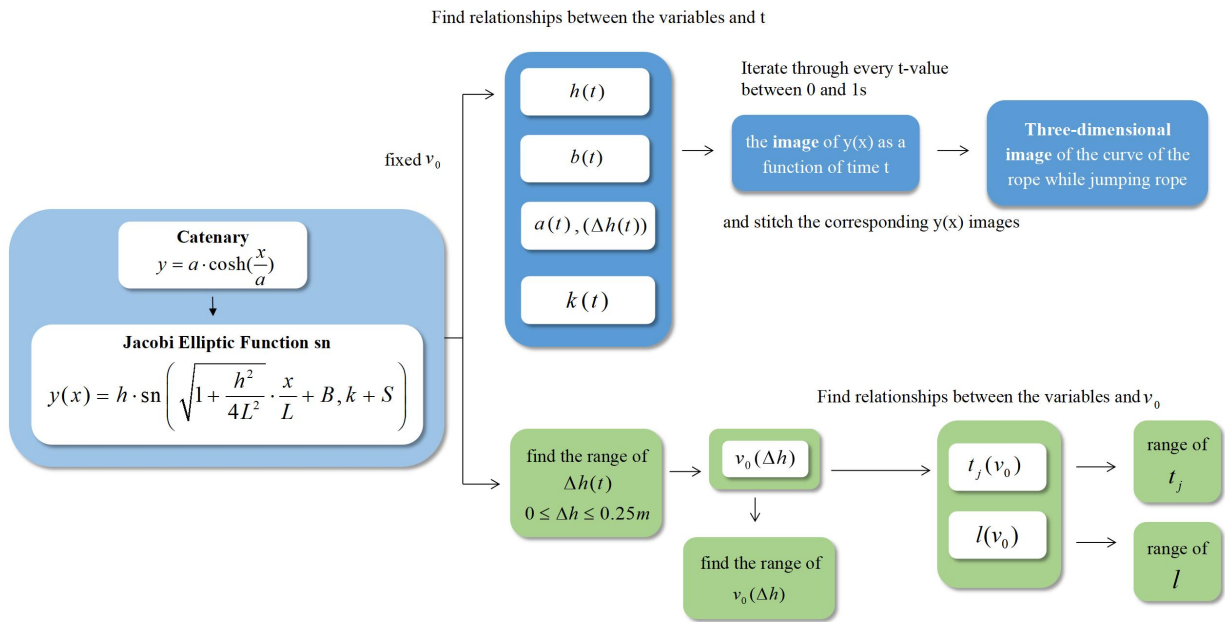


Figure 2: Jump Rope Trajectory Model General Outline

5.2 Catenary

In our team's research situation, jump rope is a kind of flexible and inextensible string. When the robot holds the rope with two hands at the same height, the rope sags naturally under its own weight and the shape of the rope is a catenary, which is shown in **Figure 3**.

The rope rotates on a fixed rotational axis. When gravity and other forces such as air resistance are neglected, the shape of the rope is completely determined by the centrifugal force. In every moment, the centrifugal force on the rope is in the plane where the rope is located in. So, this centrifugal force can be considered as a "gravity" in the rotation plane, whose direction is directly pointing out of the rope, away from the axis of rotation. This situation when the rope is rotating in the air is similar to the situation when the rope hangs under its weight. Thus, the shape of the rope is a catenary in every moment during the rotation.

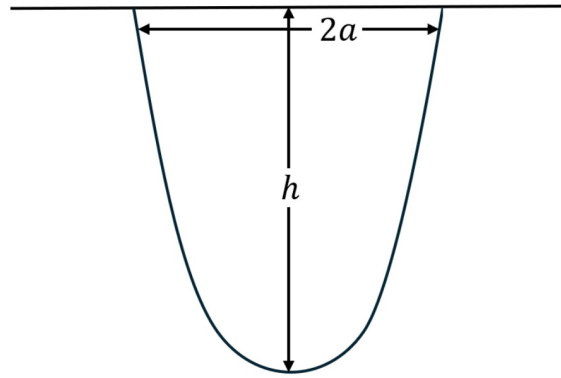


Figure 3: Shape of the Jump Rope (Catenary) When Hangs Under Two Hands

The expression of catenaries is as follows:

$$y = a \cdot \cosh\left(\frac{x}{a}\right) + s$$

5.3 Jacobi Elliptic Function (sn)

The shape of the rope can be elaborated by an expression that contains the Jacobi Elliptic Function:

$$y(x) = h \cdot \operatorname{sn}\left(\sqrt{1 + \frac{h^2}{4L^2}} \cdot \frac{x}{L} + B, k\right) + S \quad (2)$$

This function is later drawn in MATLAB to display our model.

5.4 Model Construction

In order to describe the motion of the robot and the shape of the rope during the rope-jumping process, for each variable and parameter relevant to the Jacobi Elliptic Function, our team constructs their respective functions of time.

5.4.1 Jumping Height (Δh)

Since the change in leg length during the takeoff process can be ignored, the jumping motion is similar to throwing an object directly upward under the influence of gravity. The jumping height

can be given as a function of time:

$$\Delta h(t) = v_0 t - \frac{1}{2} g t^2 \quad (3)$$

Where the gravitational acceleration $g = 9.8 \text{ m/s}^2$ and v_0 is the initial jumping velocity.

To finish constructing this function, we have to solve for v_0 . When the robot jumps to its highest position, $\Delta h = 0.25$, since the length of the arm $d = 0.25 \text{ m}$. At this moment, $a' = 0$ and arm is in a vertical position, clinging to the body. Using the formula $v_0 = \sqrt{2g\Delta h}$, we solve for $v_0 = 2.214 \text{ m/s}$. For the ease of calculations, we let $v_0 = 2 \text{ m/s}$ only in the first task.

Also, the required time for each jump from takeoff to landing t_j (jumping time) can be calculated using the formula $t_j = 2v_0/g$. This results in $t_j = 0.408 \text{ s}$. Note that the angular velocity of the rope is 1 revolution per second and the rope is still rotating in the air at $t = 0.408 \text{ s}$. This means that for $0 \leq t \leq 1 \text{ s}$, the robot is in the air during the first 0.408 seconds, waits for the rope to come back in the next 0.592 seconds, and starts the next jump at $t = 1 \text{ s}$. This is also true when $1 \text{ s} \leq t \leq 2 \text{ s}$...

Thus, the jumping height function can be derived:

$$\Delta h(t) = \begin{cases} v_0 t - \frac{1}{2} g t^2, & k \leq t \leq k + 0.408 \\ 0, & k + 0.408 < t \leq k + 1 \end{cases} \quad (k \in \mathbb{N})$$

Obviously, this is a periodic piecewise function, which is shown in **Figure 4**.

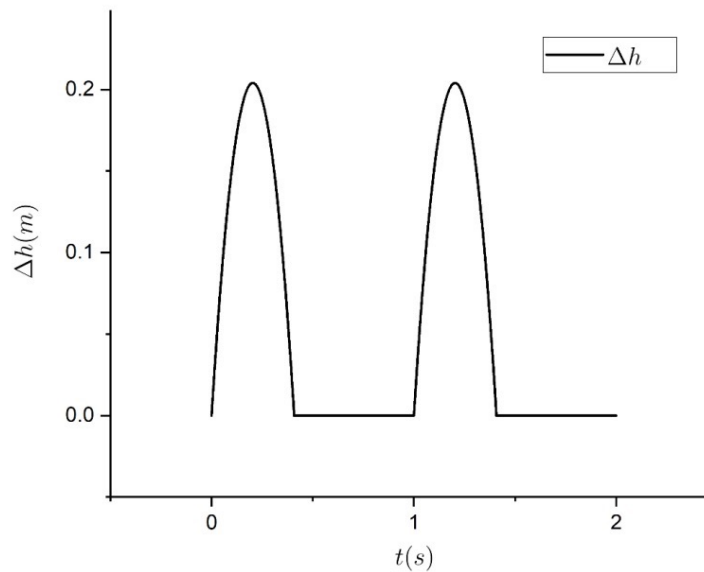


Figure 4: Graph of $\Delta h(t)$

5.4.2 Hand Distance (a)

As it is shown in Figure 1, Δh , a' , and d forms a right triangle. We can apply the Pythagorean theorem:

$$a' = \sqrt{0.25^2 - (\Delta h)^2}$$

Because $a = a' + 0.25$, we can derive the hand distance as a function of time:

$$a(t) = \begin{cases} \sqrt{0.25^2 - (\Delta h)^2} + 0.25, & k \leq t \leq k + 0.408 \\ 0.5, & k + 0.408 < t \leq k + 1 \end{cases} \quad (k \in \mathbb{N})$$

Since $a(t)$ contains $\Delta h(t)$, the function of the hand distance to time is also periodic piecewise.

5.4.3 Rope Height (h)

When Δh is increases, h increases, vice versa. So, the function of rope height to time is also periodic piecewise. Our team's approach is to construct a sinusoidal piecewise function for h

$$h(t) = A \sin(\omega t) + b \quad \text{for some value of } t$$

The minimum value of h occurs at $t = 0$ and $\Delta h = 0$. We calculate this minimum value by plugging in the coordinates of two points and finally derive $h_{\min} = 0.765$ m. Since the time for each jump from takeoff to landing is 0.408 seconds, the time for the robot to jump to its highest point is 0.204 seconds. The maximum value of h occurs at this moment. That is, $h_{\max} = 1$ m at $t = 0.204$ s. Thus, the value of A and b can be calculated accordingly. 0.408 seconds is the period of the part that the function is sinusoidal. The angular frequency ω can also be calculated accordingly.

Thus, we can derive the rope height as a function of time:

$$h(t) = \begin{cases} -0.1175 \sin(5\pi t) + 0.8825, & k \leq t \leq k + 0.408 \\ 0.765, & k + 0.408 < t \leq k + 1 \end{cases} \quad (k \in \mathbb{N})$$

5.4.4 Other parameters

The parameter L in the Jacobi Elliptic Equation has a dimension of Length, which can be calculated by the following:

$$L = \frac{2a}{\pi}$$

The parameter k in the Jacobi Elliptic Equation can be calculated by the following equation:

$$k = \sqrt{\frac{1}{1 + 4L^2/h^2}}$$

The parameter B in the Jacobi Elliptic Equation depends on where along the axis of rotation the rope is attached. It affects the horizontal shift of the overall function. Besides, variable a is the distance from the hand to the y -axis. So, there must be some correlation between these two quantities. We derive the function $B(a)$ by using linear fitting, since the best fit line is discovered to be linear after we plot many $a - b$ coordinates. The fitted function is as follows, shown in **Figure 5**:

$$B(a) = -1.497a + 2.850$$

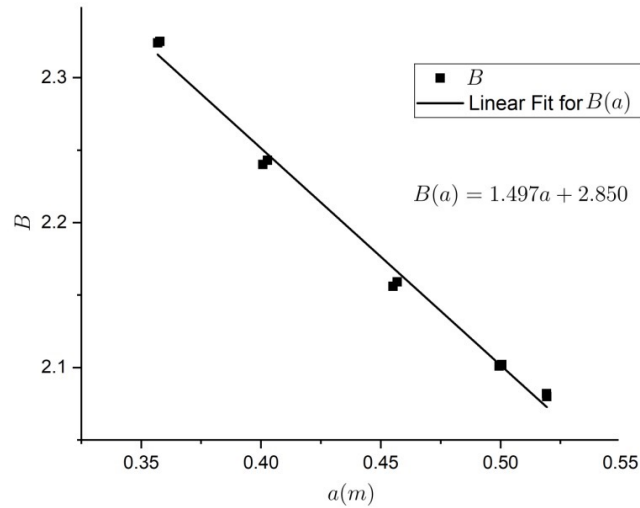


Figure 5: Linear Fit for $B(a)$

The parameter S affects the vertical shift of the overall function, it maintains that the axis of rotation is always 1 meter above the ground.

5.5 Model Results

Again, the final shape of the rope can be expressed by **Equation (2)** in part 5.3, where each variable and parameter correspond to what we derive in part 5.4. The completed expanded form of the Jacobi Elliptic Function that we derive is far too complicated and lengthy, so that it will not be displayed. But you can try to expand this function based on our team's results above.

Though the expanded form of the function will not be provided, the results will be analyzed below, both two-dimensionally and three-dimensionally.

5.5.1 2-D Results

Figure 6 below shows the curves of the rope from a 2-D perspective at $t = 0$ (start of the first jump), $t = 0.204\text{ s}$ (at the highest point), $t = 0.408\text{ s}$ and (end of the first jump), respectively.

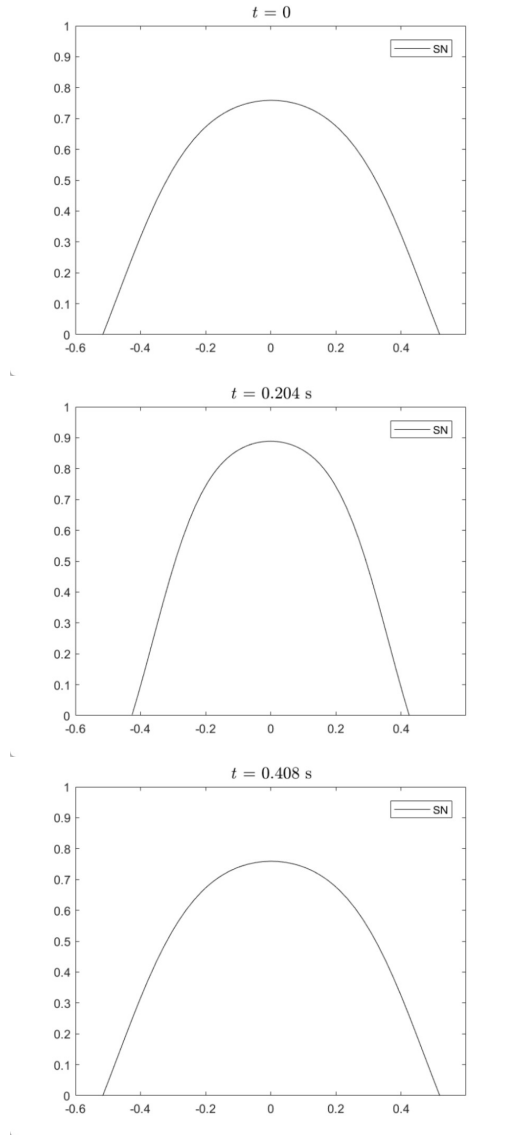


Figure 6: Shape of the Rope at $t = 0$, $t = 0.204\text{ s}$, and $t = 0.408\text{ s}$

These three graphs are adjusted for some marginal errors in the value of h . We adjust and optimize the function of $h(t)$ to achieve realistic results in reality. As shown above, at $t = 0.204\text{ s}$, when the robot is at its highest point, the shape of the rope is thinnest and has the largest vertical span compared to when the robot is on the ground (at $t = 0$ and $t = 0.408\text{ s}$). This is consistent with our theoretical analysis.

Refer to **Figure 9** in the Appendix for the original code in MATLAB. You can change relevant variables and parameters based on your need and situation.

5.5.2 3-D Results

Figure 7 below shows the rope's motion from a 3-D perspective. It shows the surface that the curve of the rope swipes (the trajectory of the rope) in a single revolution. This figure is achieved by traversing the time values from 0 to 1 second.

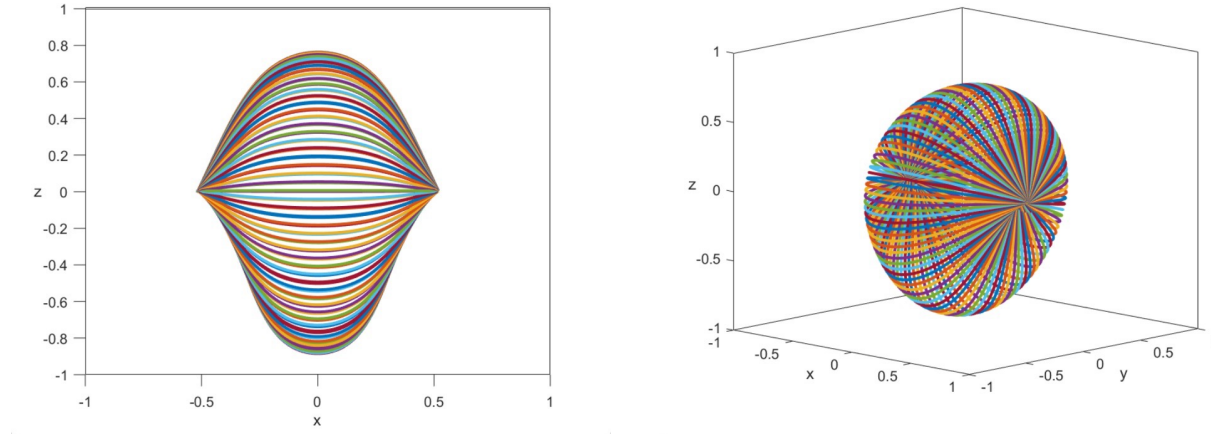


Figure 7: 3-D Trajectory of the Rope

The left image is a view perpendicular to the axis of rotation. The right image is a side view slight from the above.

6 Relevant Calculation

In the second part of our team's task, our team calculated the range of initial jumping velocity, the range of jumping time, and the range of rope length.

6.1 Range of Initial Jumping Velocity v_0

As mentioned above, $0 \leq \Delta h \leq 0.25$ m. Since $v_0 = \sqrt{2g\Delta h}$, the range of initial jumping velocity can be determined:

$$0 \leq v_0 \leq 2.214 \text{ m/s}$$

6.2 Range of Jumping Time (t_j)

Since $t_j = 2v_0/g$, the range of jumping time can be determined:

$$0 \leq t_j \leq 0.452 \text{ s}$$

6.3 Range of Rope Length (l)

The length of the rope can be calculated using the following equation:

$$l = \int_{-a}^a \sqrt{1 + (y'(x))^2} dx$$

As is mentioned in part 6.1, the initial jumping velocity will influence the maximum jumping height, and according to logic the rope length is related to maximum height. Thus, there is a specific correlation between rope length and initial jumping velocity.

However, directly analyzing this integral above is too complicated, since the function $y(x)$ is an extremely complicated expression. So, we are not able to conclude the direct relationship between l and v_0 .

Instead, our team derives the function $l(v_0)$ by using polynomial fitting. For some selected time values, we calculated the corresponding $l - v_0$ coordinates and it appears that the best fit curve is a quintic function with the highest order of 5. The fitted function is as follows, shown in **Figure 8**.

$$l(v_0) = 0.040v_0^5 - 0.124v_0^4 + 0.090v_0^3 + 0.008v_0^2 + 0.045v_0 + 1.955$$

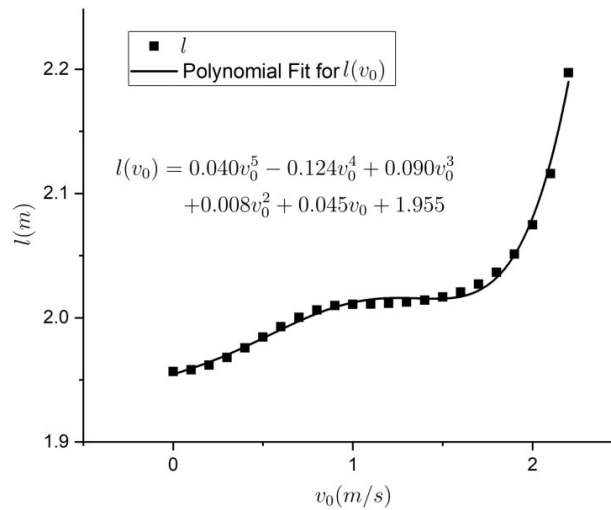


Figure 8: Polynomial Fit for $l - (v_0)$

According to the fitted curve above, the range of the rope length can be determined by the largest and smallest l -coordinates:

$$1.995 \text{ m} \leq l \leq 2.219 \text{ m}$$

7 Arm Length (d) Analysis

Besides the requirements in the two tasks, our team makes some analyses of arm length d . These include its influences on rope length l and the final shape of the rope $y(x)$.

7.1 Influence on Rope Length (l)

Since the robot's arm shifts from a horizontal position to a vertical position when it jumps to its highest position, the maximum jumping height cannot exceed the arm length. That is $0 \leq \Delta h \leq d$.

As mentioned in part 6.1, $0 \leq v_0 \leq \sqrt{2g\Delta h}$. We can replace Δh with d , which will result in $0 \leq v_0 \leq \sqrt{2gd}$.

According to the relationship we discovered between rope length and the initial jumping velocity in part 6.3, the range of rope length can be derived:

$$l(0) \leq l \leq l\sqrt{(2gd)}$$

7.2 Influence on Rope Shape ($y(x)$)

Since $t = \frac{2v_0}{g}$, we can conclude that $0 \leq t \leq 2\sqrt{\frac{2gd}{g}}$.

We let the arm length to be $d = 0.35m$, and $t = 0$ (start of the first jump), $t = 0.267s$ (at the highest point), $t = 0.534s$ (end of the first jump). **Figure 9** below shows the shape of the rope under these conditions.

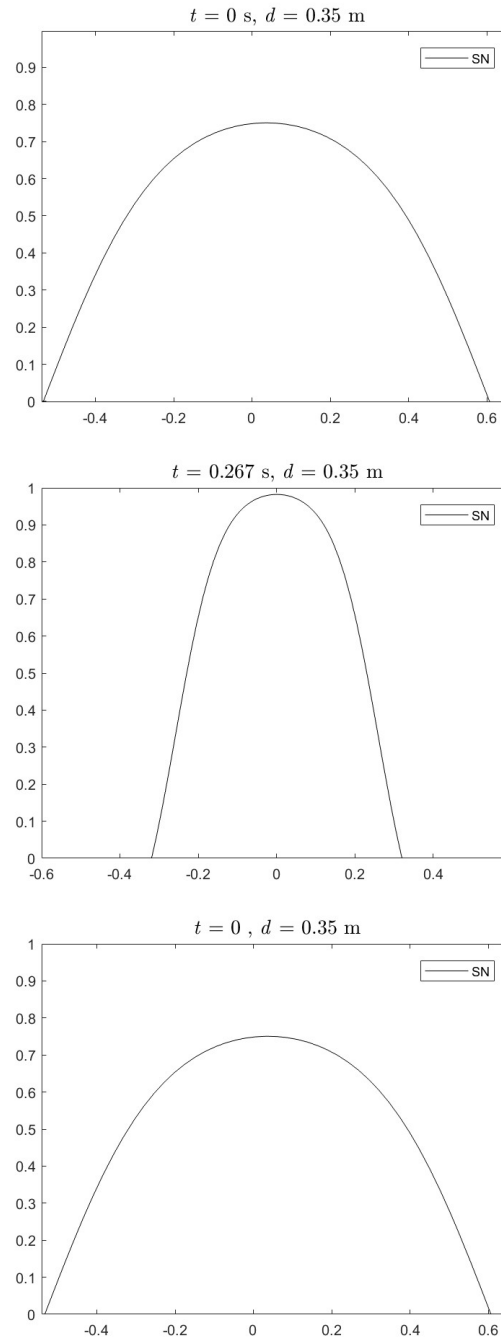


Figure 9: Shape of the Rope at $t = 0$, $t = 0.267s$, and $t = 0.534s$

Thus, the new form of the rope shape equation $y(x)$ can be derived. Just replace all the quantities related to the fixed arm length of 0.25m with the variable d . This new form of $y(x)$ is applicable to any value of arm length.

8 Conclusions

In conclusion, our team developed a model used to describe the trajectory of the rope-jumping movement. The shape of the rope at any moment is described by the Jacobi Elliptic Function.

The final function of the rope contains functions of ϕ , so that the shape of the rope at any instant can be drawn in MATLAB. You can also adjust certain variables and parameters involved in the function based on your need and situation, such as different body height or arm length.

Moreover, our team also performs some relative calculations. These include the range of initial jumping velocity, the range of jumping time, and the range of rope length. Through these numerical analyses, we are able to improve our understanding of the complete rope-jumping process.

After all, the model that we construct can help assist in IMMC's FlexRobot rope-jumping function.

References

- [1] Nordmark, A., & Essén, H. (2007). The skipping rope curve. *European Journal of Physics*, 28(2), 241–247. <https://doi.org/10.1088/0143-0807/28/2/009>
- [2] Qiu, W. (2023). 150 Physics Problems. Tsinghua University Press.
- [3] Akbar, F., Khalifah, A., & Abdullah, M. (2021). Physics model for Rope Jumping Game. *Jurnal Penelitian Pendidikan IPA*, 7(2), 281–286. <https://doi.org/10.29303/jppipa.v7i2.559>
- [4] Abbott, J. (2012). Empirical Validation of Changing the Curvature of a Jump Rope with Additional Weights.
- [5] López, R. (2024). The Hanging Chain Problem in the Sphere and in the Hyperbolic Plane. *Journal of Nonlinear Science*, 34(4). <https://doi.org/10.1007/s00332-024-10056-0>
- [6] Behroozi, F. (2014). A fresh look at the catenary. *European Journal of Physics*, 35(5). <https://doi.org/10.1088/0143-0807/35/5/055007>
- [7] Aristoff, J., & Stone, H. (2011). The aerodynamics of jumping rope. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 468(2139), 720–730. <https://doi.org/10.1098/rspa.2011.0389>

Appendices

The figure below is the original code in MATLAB for the Jacobi Elliptic Function used to describe the shape of the rope. You can change relevant variables and parameters based on your need and situation.

```
classdef part1 < matlab.apps.AppBase
    properties (Access = public)
        UIFigure          matlab.ui.Figure
        UIAxes             matlab.ui.control.UIAxes
        Slider             matlab.ui.control.Slider
        SliderT            matlab.ui.control.Slider
        LabelB             matlab.ui.control.Label
        LabelT             matlab.ui.control.Label
    end

    methods (Access = private)

        function SliderValueChanged(app, ~)
            b = app.Slider.Value;
            t = app.SliderT.Value;

            syms x;
            if t <= 0.4082
                h_1 = 2*t-4.9*t.^2;
                h = -0.1175*cos(5*pi*t)+0.8825;
            elseif t > 0.4082
                h_1 = 0;
                h = 0.765;
            end
            a = (0.0625-(h_1).^2).^0.5+0.25;
            L = 2*a/pi;
            k = (1/(1+(4*L.^2)/h.^2)).^0.5;
            u = ((1+h.^2/(4*L.^2)).^0.5)*x/L-3.6*a+3.98;
            s = h*ellipj(((1+h.^2/(4*L.^2)).^0.5)*a/L-3.6*a+3.98,k);

            [S] = h*ellipj(u,k)-s;
            fplot(app.UIAxes,S,[-a,a]);
            legend(app.UIAxes,'SN','Location','best');
            grid(app.UIAxes,'on');
            title(app.UIAxes,'Jacobi Elliptic Function sn');
            xlim(app.UIAxes, [-0.6 0.6]);
            ylim(app.UIAxes, [-1.1 1.1]);
            app.LabelB.Text = sprintf('b = %.4f', b);
            app.LabelT.Text = sprintf('t = %.4f', t);
        end
    end
end
```

```

        end
    end
    methods (Access = private)
        function createComponents(app)
            app.UIFigure = uifigure('Visible', 'off');
            app.UIFigure.Position = [100 100 640 480];
            app.UIFigure.Name = 'UI Figure';
            app.UIAxes = uiaxes(app.UIFigure);
            app.UIAxes.Title.String = '';
            app.UIAxes.Position = [112 114 411 302];

            app.Slider = uislider(app.UIFigure);
            app.Slider.Limits = [0, 5];
            app.Slider.Position = [194 70 260 3];
            app.Slider.ValueChangedFcn = createCallbackFcn(app,...
                @SliderValueChanged, true);

            app.SliderT = uislider(app.UIFigure);
            app.SliderT.Limits = [0, 1];
            app.SliderT.Position = [194 30 260 3];
            app.SliderT.ValueChangedFcn = createCallbackFcn(app,...
                @SliderValueChanged, true);

            app.LabelB = uilabel(app.UIFigure);
            app.LabelB.Position = [120 50 80 22];
            app.LabelB.Text = sprintf('b = %.4f', app.Slider.Value);

            app.LabelT = uilabel(app.UIFigure);
            app.LabelT.Position = [120 10 80 22];
            app.LabelT.Text = sprintf('t = %.4f', app.SliderT.Value);

            app.UIFigure.Visible = 'on';
        end
    end
    methods (Access = public)
        function app = part1()
            createComponents(app);
            registerApp(app, app.UIFigure);
            if nargin == 0
                clear app
            end
        end
        function delete(app)
            delete(app.UIFigure);
        end
    end
end
end

```

Figure 10: Original MATLAB Code for the Jacobi Elliptic Function

The figure below is the original code in MATLAB for the Jacobi Elliptic Function used to draw the trajectory of the rope three-dimensionally.

```

function u5()
    close all; clc;

    tRange = linspace(0, 1, 101);
    nLines = length(tRange);

    figure('Name','S(x,t) - 101 lines, each +3.6°','NumberTitle','off');
    set(gcf, 'Color', 'w');
    hold on; grid on; box on;

    cm = lines(nLines);

    max_aVal = 0;

    for i = 1 : nLines
        tVal = tRange(i);

        aVal = aFunc(tVal);

        if aVal > max_aVal
            max_aVal = aVal;
        end

        xNum = 200;
        xVec = linspace(-aVal, aVal, xNum);

        SVec = arrayfun(@(xx) S_func(xx, tVal), xVec);

        yVec = zeros;

        yVec = zeros;

        alphaDeg = 108 + (i - 1) * 3.6;
        alphaRad = deg2rad(alphaDeg);
        % x' = x
        % y' = y*cos(a) - z*sin(a)
        % z' = y*sin(a) + z*cos(a)
        yRot = yVec*cos(alphaRad) - SVec*sin(alphaRad);
        zRot = yVec*sin(alphaRad) + SVec*cos(alphaRad);

        plot3(xVec, yRot, zRot, ...
            'LineWidth', 2, ...
            'Color', cm(i,:), ...
            'DisplayName', sprintf('Line %d: t=%.3f', i, tVal));

        xlim([-aVal, aVal]);
    end

    xlim([-2,2]);
    ylim([-2,2]);
    zlim([-2,2])

    xlabel('x');
    ylabel('y_{rot}');
    zlabel('z_{rot}');
    title('0~0.5, 101 lines, each rotated 3.6° about X-axis (No Spiral)');
    view(45, 25);
end

```

```

end

function val = h1Func(t)
    if t >= 0 && t <= 0.4082
        val = 2*t - 4.9*t^2;
    else
        val = 0;
    end
end

function val = aFunc(t)
    h1_val = h1Func(t);
    val = sqrt(max(0, 0.0725 - h1_val^2)) + 0.25;
end

function val = hFunc(t)
    if t >= 0 && t <= 0.4082
        val = -0.0085*cos(5*pi*t) + 0.6385;
    elseif t > 0.4082 && t <= 1
        val = 0.630;
    else
        val = 0;
    end
end

function S_val = S_func(x, t)
    a_ = aFunc(t);
    h_ = hFunc(t);
    L_ = 1.92582*a_ / pi;

    if abs(h_) < 1e-14
        k_ = 0;
    else
        k_ = sqrt(1 / (1 + (4*L_^2)/(h_^2)));
    end

    A = sqrt(1 + h_^2/(4*L_^2));
    u_ = A*(x/L_) - 1.49745*a_ + 2.85037;

    tempArg = A*(a_/L_) - 1.49745*a_ + 2.85037;
    s_ = h_ * ellipj(tempArg, k_);

    S_val = h_ * ellipj(u_, k_) - s_;
end

function snVal = ellipj(u, k)
    snVal = jacobiSN(u, k);
end

```

Figure 12: Original MATLAB Code for drawing the 3-D Rope Trajectory